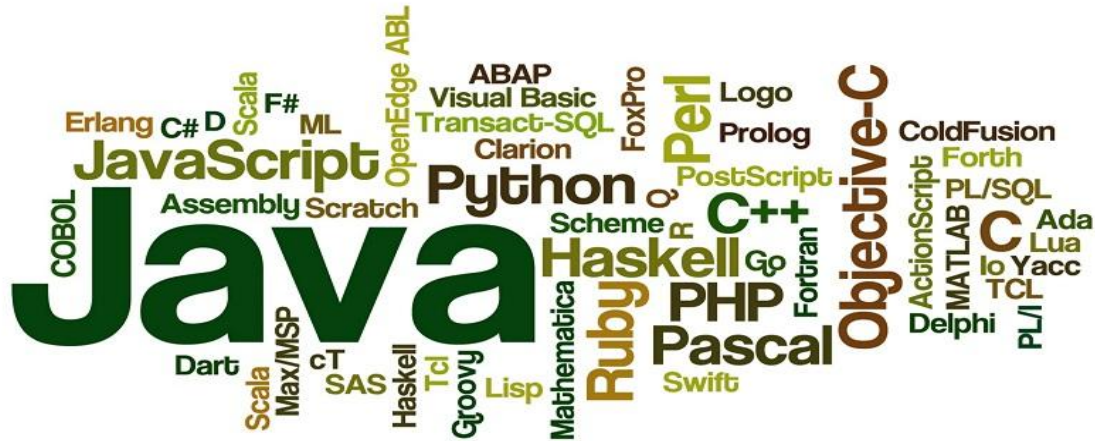




TECNICATURA UNIVERSITARIA EN PROGRAMACIÓN

INGLES II

Docentes: Virginia Vallina y Nancy Rotchen.

MÓDULO 1

C++

C++ ("C Plus Plus) is a general-purpose programming language. It is regarded as a *middle-level* language, because it comprises a combination of both high-level and low-level language features. It was developed by Bjarne Stroustrup in 1979 at Bell Labs as an enhancement to the C programming language and originally named "C *with classes*". It was renamed to C++ in 1983.

C++ is widely used in the software industry. Some of **its** application domains include systems software, device drivers, embedded software, high-performance server and client applications, and entertainment software such as video games. Several groups provide both free and proprietary C++ compiler software, including the GNU Project, Microsoft, Intel, Borland and others.

The language began as enhancements to C, first adding classes, then virtual functions, operator overloading, multiple inheritance, templates, and exception handling among other features. After years of development, the C++ programming language standard was ratified in 1998 as ISO/IEC 14882:1998. The current standard is the 2003 version, ISO/IEC 14882.2003. The next standard version (known informally as C++0x) is in development.

C++ is a statically typed, free form, multi-paradigm, compiled language where compilation creates machine code for target machine hardware. C++ inherits most of C's syntax and the C preprocessor.

C++ provides more than 30 operators, covering basic arithmetic, bit manipulation, indirection, comparisons, logical operations and more. Almost all operators can be overloaded for user-defined types, with a few notable exceptions such as member access. The rich set of overloadable operators is central to using C++ as a domain specific language. The overloadable operators are also an essential part of many advanced C++ programming techniques, such as smart pointers.

Overloading an operator does not change the precedence of calculations involving the operator, nor does **it** change the number of operands that the operator uses.

Conteste las siguientes preguntas con información proveniente del texto:

1. ¿Cuál es el tópico del texto?
2. ¿Qué tipo de texto es?
3. ¿A qué se refieren los siguientes índices tipográficos?
 - a. "C Plus Plus":.....
 - b. 1979:
 - c. "C with Classes":
 - d. 1983:
 - e. 30:
4. ¿Qué características posee este lenguaje de programación?
5. ¿Dónde se lo utiliza ampliamente?
6. ¿Quién inventó C++ y con qué fin lo hizo?
7. ¿Cuándo se le puso el nombre C++?
8. En el primer párrafo hay una definición. Transcríbela e indique de qué tipo es y por qué. Marque sus partes.
9. En el segundo párrafo hay una ejemplificación. Subráyela. ¿Cuál es la función?
¿Qué conector se usó?
10. En el segundo párrafo hay un conector de adición. ¿Cuál es?
11. El conector "*because*" (línea 2) establece una relación lógica causal:
 - a. ¿cuál es la causa?.....
 - b. ¿cuál es el efecto?.....
12. El conector "*such as*" (línea 8) introduce un ejemplo:
 - a. ¿Cuál es el ejemplo?.....
 - b. ¿Qué concepto se ejemplifica?.....
13. ¿A qué se refieren los siguientes términos?:
 - a. "*its*" (línea 6):
 - b. "*it*" (línea 26):
14. Traslade al castellano los bloques nominales subrayados en el texto.

La comparación en inglés

El concepto de comparación es una operación lógica del pensamiento que consiste en la confrontación de objetos, hechos o fenómenos con el fin de poner de manifiesto los rasgos de semejanza o de diferencia entre ellos o ambos a la vez.

La comparación puede ser por diferencia o contraste (superioridad, inferioridad) si lo que se resaltan son las diferencias, o bien por igualdad y por analogía si se resaltan las semejanzas.

A. Comparación por diferencia o contraste

	Adjetivo base	Formas comparativas	Formas superlativas
ADJETIVOS REGULARES:			
Con 1 sílaba	Small Big cold	smaller than bigger than colder than	the smallest the biggest the coldest
Con 2 sílabas	Funny happy gentle modern	funnier than happier than more gentle than more modern than	the funniest the happiest the most gentle the most modern
Con 3 ó más sílabas	dangerous interesting	more/less dangerous than more/less interesting than	the most/ the least dangerous the most/ the least interesting
ADJETIVOS IRREGULARES:			
	bad good little many (Count) much (Unc.) far (lejos)	worse than better than less/lesser than more than more than -farther (physical distance) (than) -further (figurative distance) (than)	the worst the best the least the most the most -the farthest/furthest

B. Comparación por igualdad y por analogía (similar)

BN 1 + VERBO (forma afirmativa o negativa) + **AS** + **ADJECTIVO** + **AS** + BN 2
BN 1 + VERBO (forma afirmativa o negativa) + **LIKE** + BN 2

Ejemplos:

- The new software programme is **easier than** its old version.
- Samsung Galaxy S10 Plus is, simply put, **the best** Android phone in the world.
- The iPhone 8 Plus isn't **as exciting as** the iPhone 11 but it's still **faster than** others.
- "Counter-Strike: Global Offensive" was ranked one of **the most popular** PC games in 2019.
- Many malware attacks are becoming **more harmful** and take advantage of software vulnerabilities in common applications, like operating systems and browsers.
- HP has just released **the latest** version of LaserJet Pro Series printers.
- She wants to code **like** a professional.

COMPARATIVE	SUPERLATIVE
<i>EQUALITY</i> AS + adjective + AS (tan ... como)	THE MOST + (long) adjective + noun (el más ...)
<i>INFERIORITY</i> LESS + adjective + THAN (menos ... que)	
<i>SUPERIORITY</i> MORE + (long) adjective + THAN (más ... que) (short) adjective -ER + THAN (más ... que)	THE + (short) adjective -EST + noun (el más...)

C. La doble comparación:

THE + comparativo, THE + comparativo = (cuanto más + adjetivo, más + adjetivo)

- The more** you exercise, **the better** you'll feel.
- The longer** the program, **the more complex** is the job of the compiler.
- The faster** you drive, **the more dangerous** it is.

D. Otra construcción de comparación:

Comparativo + and + comparativo = "cada vez más + adjetivo"

- Ejemplos: - bigger **and** bigger = cada vez más grande
- younger **and** younger = cada vez más joven
 - The Internet is more **and** more interesting = cada vez más interesante
 - Flight tickets are becoming **more and more expensive** - *Los billetes de avión son cada vez más caros.*
 - The weather is getting **hotter and hotter**.

ACTIVIDADES

1. **Traslade al castellano las siguientes oraciones e indique qué tipo de comparación se presenta en cada una de ellas.**
 - 1) A program that converts a more abstract (high-level) language into machine code is called a **compiler**.
 - 2) The sales figures are as bad as predicted; the company will probably go bankrupt.
 - 3) The imperative programming paradigm was one of the earliest and was developed using machine language.
 - 4) **Functional programming** is essentially less complex and offers better readability than imperative.
 - 5) Coding with programming languages is relatively difficult as many lines of code are required for a single function. Creating a code function with a scripting language is easier as it requires only a few short and specific lines to be written.
 - 6) The more I practise English, the fewer mistakes I make.
 - 7) When it comes to the differences between natural language and programming language, it is worth noting that the latter is stricter and less tolerant than the former.
 - 8) The best languages to do UI (User Interface) prototyping are Java, C++, Python and C#. They have the best and most versatile tools and libraries for user interface purposes.

- 9) Assembly code is most useful when performance is the most important goal.
- 10) C is often selected because it is more compact than C++ and runs faster. It's the second most common programming language following Java.
- 11) I'm almost as good in Maths as in Science.
- 12) If C++ is so similar to C, but it's more complex and offers more, why bother with C?
- 13) These exams are getting worse and worse every year.

2. Complete las siguientes oraciones con la forma comparada de los adjetivos entre paréntesis.

- a. UTN offers multiple technical resources for all programmers and it has some of (modern) laboratories in the country.
- b. Which is (good) university in Argentina?
- c. Some private universities are (cheap) others.
- d. Where is (old) university in Europe?
- e. Which is (difficult) to study, Physics or Maths?

LA MODALIDAD EN EL TEXTO TÉCNICO

Observe las siguientes oraciones extraídas de un texto sobre los temas que un programador debería saber.

1. This list ***provides you with*** a good starting point.

- If you want to become a programmer, then you **should know** data structure and algorithms well.
- This is one of the important topics of any programming job interview and without you knowing basic data structures (e.g. array, linked list, map, set) **you can't write** a real-world application.
- That's why every programmer **should put** serious effort to learn data structures and algorithms during their computer science course.
- Even If you are a self-taught programmer, then also **you must know** data structures and algorithms.

Observe los verbos resaltados en negrita. Y luego, responda las preguntas:

- ¿En qué tiempo verbal se encuentra la oración 1?
- ¿Qué diferencia nota entre la acción de la primera y las restantes?
- ¿Qué nuevos elementos gramaticales hay en las oraciones 2, 3, 4 y 5?
- ¿Qué nombre reciben estos elementos? ¿Qué valor o modo le agregan a la acción en esas oraciones?

Complete el siguiente cuadro y escriba el equivalente en castellano:

Oración 2: <i>should know</i>	<i>Debería saber.</i>
Oración 3:	
Oración 4:	
Oración 5:	

VERBOS MODALES: MODAL VERBS

Mediante la modalidad, los interlocutores proyectan una intervención o una actitud personal en el contenido de la proposición, a través de nociones como: certidumbre, posibilidad, probabilidad, necesidad, volición, obligación, permiso, duda, advertencia, prohibición, etc. Estas nociones expresan una construcción de la realidad de acuerdo con las actitudes de los interlocutores. Los verbos modales actúan como verbos auxiliares, pero a diferencia de otros verbos auxiliares como "do", "have" y "be", los verbos modales sirven para modalizar al verbo principal de la oración, indican la actitud del hablante. Los verbos "be", "do" y "have" difieren de los verbos modales en el sentido en que pueden actuar como verbos principales.

Características de los verbos modales:

Desde el punto de vista gramatical, los verbos modales poseen las siguientes características:

- No toman las terminaciones de los participios (presente y pasado) y no existe concordancia con el sujeto. De modo que decimos "*He may go to the workshop*"
- Forman la negación agregándole la palabra "*not*" detrás. Por ejemplo: "*You might not like that*"
- Para la forma interrogativa, realizan inversión con el sujeto de la oración. Por ejemplo: "*Can I have another biscuit?*", "*Would you like to go to the cinema?*"

4. Detrás de un verbo modal, siempre se utiliza un verbo en infinitivo sin el “to” delante.
Por ejemplo: “I will call you tomorrow”

A continuación, lea la lista de verbos en inglés que comunican diferentes modalidades:

Can – could – may – might – must – should – will – would – have to – need to.

A continuación, lea la lista de verbos modales con sus significados:

CAN: “poder/puedo”	Habilidad/Posibilidad/solicitud/capacidad
MAY: “poder/puedo”	Posibilidad
SHOULD: “deberia”	Advertencia/Recomendación/Sugerencia/consejo/ “debe”
WILL: “lo hare”	Probabilidad/Certeza/Promesa/predicción
MUST: “deber”	Obligación/necesidad/suposición/inferencia
HAVE/HAS TO: “tiene que/debe”	Obligación
MIGHT: “podría”	Posibilidad remota
COULD: “podría/pudo”	Posibilidad/sugerencia/habilidad en pasado
NEED TO	Necesidad urgente
MUSTN'T: “no debe”	Prohibición
CAN'T: “no puede”	Imposibilidad/Prohibición/incapacidad
WOULD: “...ía”	Situación hipotética/Suposición/preferencia

Actividades

1. **Elija la opción más apropiada para cada situación y justifique su elección.**
 - a. If you want to be a good student of Engineering, you MAY/HAVE TO study hard.
 - b. Because of space limitations, there NEED TO/CAN'T be more than 30 students per classroom.
 - c. With early registration, students CAN/CAN'T get a vacancy.
 - d. Governments SHOULD/MIGHT gather and design a global plan to help the poor.
 - e. Students WOULD/MUSTN'T become fluent speakers of English if they practiced the language every day.

2. Traslade las siguientes oraciones al castellano. Indique la modalidad del verbo en cada una de ellas.

- 1) Programming language defines a specific set of instructions that can be used to produce various kinds of output.
- 2) The assembly program must be translated into machine language before it can be executed by the computer. This is done by using a program called an assembler.
- 3) Scripting languages are generally interpreted. A primary focus of a scripting language does not build the application but might provide behavior to an existing application.
- 4) If a person learns C programming first, it will help him to learn any modern programming language.
- 5) A compiler is a piece of software that translates the source code written in a high-level programming language to object code in a low-level programming language. This process is performed so that the code may be transformed into an executable program.
- 6) Most programmers like Java because it allows them to write applets, which makes web pages more interactive and attractive. They can create graphical objects and new controls.
- 7) A high-level language has to be translated into the machine language by translator and thus a price in computer time is paid.
- 8) Every programmer, especially beginner, should spend some time in a learning text editor and some keyboard shortcut to becoming a smart and productive developer.

THE HISTORY OF C LANGUAGE

The C programming language was devised in the early 1970s by Dennis M. Ritchie an employee from Bell Labs (AT&T).

In the 1960s Ritchie worked, with several other employees of Bell Labs (AT&T), on a project called Multics. The goal of the project was to develop an operating system for a large computer that could be used by a thousand users. In 1969, AT&T (Bell Labs) withdrew from the project, **because** the project could not produce an economically useful system. **So** the employees of Bell Labs (AT&T) had to search for another project to work on (mainly Dennis M. Ritchie and Ken Thompson).

Ken Thompson began to work on the development of a new file system. He wrote, a version of the new file system for the DEC PDP-7, in assembler. (The new file system was also used for the game Space Travel). Soon they began to make improvements and add expansions. (They used their knowledge from the Multics project to add improvements). After a while a complete system was born. Brian W. Kernighan called the system UNIX, a sarcastic reference to Multics. The whole system was still written in assembly code.

Besides assembler and Fortran, UNIX also had an interpreter for the programming language B. (The B language is derived directly from Martin Richards BCPL). The language B was developed in 1969-70 by Ken Thompson. In the early days, computer code was written in assembly code. To perform a specific task, you had to write many pages of code. A high-level language like B made it possible to write the same task in just a few lines of code. The language B was used for further development of the UNIX system. Because of the high-level of the B language, code could be produced much faster than in assembly.

A drawback of the B language was that it did not know data-types. (Everything was expressed in machine words). Another functionality that the B language did not provide was the use of "structures". The lag of these things formed the reason for Dennis M. Ritchie to develop the programming language C. Therefore, in 1971-73 Dennis M. Ritchie turned the B language into the C language, keeping most of the language B syntax while adding data-types and many other changes. The C language had a powerful mix of high-level functionality and the detailed features required to program an operating system. **Therefore**, many of the UNIX components were eventually rewritten in C (the UNIX kernel itself was rewritten in 1973 on a DEC PDP-11).

The programming language C was written down, by Kernighan and Ritchie, in a now classic book called "The C Programming Language, 1st edition".

For years the book "The C Programming Language, 1st edition" was the standard on the language C. In 1983, a committee was formed by the American National Standards Institute (ANSI) to develop a modern definition for the programming

language C (ANSI X3J11). In 1988, they delivered the final standard definition ANSI C.

<http://www.codingunit.com/the-history-of-the-c-language>

1) Tópico del texto:

.....

2) Tipo de texto:

.....

3) Fuente textual:

.....

4) ¿Quién inventó este programa, cuándo y dónde?

.....

.....

.....

5) ¿Qué era "Multics"? Explique brevemente.

.....

.....

6) ¿Qué rol cumplió Ken Thompson en el proyecto?

.....

.....

.....

7) ¿Qué es UNIX? ¿Cómo surgió y con qué fin?

.....

.....

8) Comente brevemente sobre la historia de este programa teniendo en cuenta lo expresado en el texto:

1960's:

.....

.....

1969:

.....

.....

1969-70:

.....

.....

1971-73:

.....

.....

1983:

.....

.....

1988:

.....
.....

9) Analice los conectores resaltados en negrita. ¿Qué tipo de conectores son y qué dos ideas unen?

- a. _____

- b. _____

- c. _____

10) En el segundo párrafo hay dos verbos modales y en el último párrafo hay uno. ¿Cuáles son y qué están expresando?

- a. _____
- b. _____
- c. _____

11) ¿Qué se compara en el 4ª párrafo?

.....
.....
.....

CV en Ingles (CV in English):

¿Qué es un CV (curriculum) en inglés y cuál es su estructura en inglés?

- El *curriculum vitae* (CV) en inglés, también llamado *resume*, es junto a la carta de presentación (Cover letter), el documento que debes enviar como parte de una solicitud de empleo. Cuando el equipo de Recursos Humanos lo reciba, podrá saber de un vistazo cuál ha sido tu trayectoria hasta la fecha y valorar si mereces seguir adelante en el proceso de selección.
- En un CV de ingeniería es importante demostrar las habilidades y competencias que puedes aportar a la compañía como profesional. La manera en que presentes tu CV dice mucho de tu personalidad y tu forma de trabajar. Cuida detalles como la limpieza, el orden, la disposición de la información, la calidad de la redacción y la falta de errores. Para atraer la atención de los gerentes de contratación, debes evitar cualquier elemento o información que no aporte nada a quien evalúa tu perfil.

- Al igual que sucede en castellano, el currículum en inglés debe seguir una estructura determinada para facilitar la lectura de la persona que gestione la oferta de trabajo a la que aspiras. Además, deberás organizar tu información separándola en secciones claramente diferenciadas e incluir tanto los apartados obligatorios, como los adicionales que necesites según tu situación personal y el país en el que quieres trabajar.

La estructura básica de un CV en inglés es la siguiente. Leala e indique que información incluiría en cada sección. Luego vea el ejemplo del CV y decida en grupos qué le parece correcto y qué considera que podría mejorarse.

- Personal info (Datos de contacto)
- Personal statement (Descripción personal)
- Working experience (Experiencia)
- Education (Educación)
- Skills (Habilidades)
- Languages (Idiomas)
- Otras secciones (Referencias, proyectos, licencias, etc.)

Personal information:

Name: María Quintana

Address: Avda. Séneca 5, Madrid 28040

Telephone: 00 34 91 5435201

Email: mquintana0782@telefonica.net

Date of birth: 28/07/82

Education and training:

2006	Online diploma in web-based technology for business.
2005	Course in web design at the Cybernetics College, London: HTML, Java and Macromedia Dreamweaver.
2004	Course in computer hardware and networking at the Cybernetics College, London.
1999-2004	Degree in Computer Science and Engineering, University of Madrid.

Work experience:

January 2006- present	Part-time Webmaster at www.keo.es ; responsible for updating
May 2005-December 2006	The site and using Adobe Flash to create animations. IT Consultant at Media Market, specializing in e-commerce and IT

Strategies.

IT skills:

Knowledge of multiple computer platforms (Windows, Mac and Linux); strong database skills (including the popular open source MySQL database); complete understanding of graphics formats and cascading Style Sheets.

Personal skills:

Social and organizational skills.
Good communication skills.

Languages:

Spanish mother tongue; English; Arabic.

Hobbies and interests:

Web surfing, listening to music and travelling.

Tips

1. Start with a short profile to show who you are and what you offer.
2. List your employment history, starting with the present.
3. Explain any gaps, for example time spent travelling or having children.
4. Use bullet points for your main responsibilities and experience in each role.
Choose the aspects of your previous jobs that are most relevant to the new job you are applying for.
5. Then give your educational background. List the relevant qualifications that you have, starting with the most recent. Include the title of your qualification, where you studied and the date you successfully completed it.
6. Consider putting Education above Employment if you don't have a lot of work experience yet.
7. List other experiences or interests you may have if they are relevant to the job.
8. If you have someone who could support your application, add their contact details or say *References available on request* at the end.

Are the sentences true or false?

	Answer	
1. It is a good idea to start with a short summary about you.	True	False

2. You should write your work history in order, with your most recent job at the end. True False
3. You should list all your responsibilities in detail. True False
4. You should give the full title of your qualifications, with the date you passed each one and the organisations that awarded them to you. True False
5. It is a good idea to include hobbies if they are relevant to the job. True False
6. Include references to support your application if you can. True False

Put the details in the correct groups.

Responsible for production in a reputable Seoul-based design firm	Led a team of designers to develop graphic and production materials	MA in Creative Arts and Design, Leeds Arts University (July 2011)
Developed over 200 graphic design projects	Diploma in Graphic Design, York College (July 2008)	Proficient with Microsoft Office, Adobe Photoshop and Sketch
High levels of critical thinking, creativity and problem- solving skills	Excellent communicator who brings friendliness, confidence and empathy to leadership and delegation skills	BA in Philosophy, Politics and Economics, University of Oxford (June 1998)

Skills	Education	Work experience – Chief Graphic Designer, BEE&BONN Ltd

Activity

Write the CV writing tips in the correct group.

use unusual fonts or colours	detail your education and work experience
make a ten-page-long CV	use an unprofessional photo
mention your language skills	highlight your strengths that are relevant to this job

Do	Don't

Which tips do you think are the most useful for writing a good CV? Add some of your own
What tips do you have for preparing a CV?

¿Qué le agregaría usted a este CV? ¿Foto? ¿Proyectos? ¿Más información sobre algo en particular?

Ahora mire los vídeos compartidos en Teams y realice su propio CV. No use plantillas de Canvas o las típicas de internet. Puede usar uno de los modelos compartidos de Teams o el del cuadernillo.

LA ENTREVISTA LABORAL:

JOB INTERVIEW

Lea el diálogo que sigue. Preste especial atención a las preguntas realizadas.

Human Resources Manager: Tell me a little about yourself.
Applicant: I am a senior student of programming at the National Technological University looking for a senior position in order to continue my professional development.
HRM: Can you tell me about your previous job experience?
A: I have been an intern in BELO a crypto-currency wallet.
HRM: What have you learnt from your experience there?
A: Firstly, I have discovered a passion for App development and programming. I have also learnt the principles of team building and more people skills.
HRM: What are your weaknesses and strengths?
A: Well, I am a responsible proactive employee who is always looking to excel at whatever I embark on and precisely because of this tendency, I can tend to be a bit competitive or too obsessive with perfection and that can be a liability at times.
HRM: Where would you like to be in 5 years?
A: I picture myself as a senior developer in a multinational company such as yours.

ACTIVIDADES

1. Mire el video titulado: **Top Interview Tips: Common Questions, Nonverbal Communication & More Indeed** que se encuentra en el campus o en el siguiente link:
<https://www.youtube.com/watch?v=HG68Ymazo18&t=1s>

2. Transcribe 4 recomendaciones que considere principales de las que se mencionan y trasládelas al español.

3. Identifica en el video algunas de las preguntas comunes en una entrevista laboral y trasládalas al español.

1. Lea las siguientes preguntas más frecuentes en una entrevista laboral. Trasládelas al castellano.

a) Please tell me a little about yourself.

.....

b) What have you learned from your past jobs?

.....

c) Why did you leave your last job?

.....

d) Tell me about your last position and what you did.

.....

e) Where would you like to be in 5 years from now?

.....
f) Why are you applying for this post?

.....
g) What are the key tasks for an Electronics engineer?

.....
h) What are your strengths and weaknesses?

.....
i) What are the top 3 skills for an Electronics engineer?

.....
j) How would you measure your job performance of your position?

.....
k) What do you know about this company?

.....
l) Can you describe two or three major trends in your area?

.....
m) What qualifications have you attained?

.....
n) What is the skill you have learned recently?

.....
o) Why do you want this job?

1. ¿Cuáles otras preguntas crees que pueden realizarse en una entrevista de trabajo?

.....
.....
.....
.....

2. Combina las preguntas que siguen con las respuestas. La primera está resuelta.

1) Can you describe yourself? **A**

2) What qualifications have you attained?

3) What do you know about this company?

4) Why are you the right candidate for this position?.....

5) What do you do in your free time?

6) What does success mean to you?

- 7) What are your salary expectations?
- 8) Do you want to ask anything from us?
- a) I am a hardworking and detail-oriented person. I love being part of a team and learning new things. ✓
- b) I know that it is the fastest growing company in the sector, and it offers great benefits and promotion opportunities to its employees.
- c) First, I graduated from High School as an electrical technician and now I am attending my last year of programming at UTN.
- d) Salary is not a definite criterion for me, but it needs to be according to the present industry norms.
- e) Success is a win-win situation. I would like to take myself and my team along my success path because I believe success cannot be an individual goal but rather a team objective.
- f) I enjoy kayaking, plane modelling and on the weekends, I go out with friends.
- g) Because I have worked in this sector in my previous position in Accenture and I feel I could improve the area due to my expertise and willingness to collaborate with the team.
- h) I would like to know what you like best about this company.

Entrevista laboral/ job interview

Job interviews can be a nerve-racking experience, especially when you don't feel prepared for them. And with the variety of interview questions that can be asked these days, it's hard to know what type of questions you should prepare for.

Here are three types of interview questions that you should practise answering before that important job interview.

1. Common interview questions

Tell us about yourself.

What are your strengths and weaknesses? Why do you want to leave your current job?

These questions are for the interviewer to get to know you and to see if you're the best person for the job. Don't simply list things like your hobbies, your strengths or your work experience. Instead, give examples and use them to show your personality and the characteristics you have that make you perfect for the job. Your interviewer may want to ask questions about certain areas of your CV, so use this opportunity to link your experience to the job you're applying for.

Avoid: Giving a detailed life history or telling long stories that are irrelevant to the job or to the company.

2. Competency questions

Tell me about a time you had to work as a team.

Tell me about a time you had to use your creativity to solve a problem.

Tell me about a time when you experienced conflict with a colleague and how you handled that situation.

Here, the candidate is asked questions about situations they have faced in the past that can demonstrate a particular skill they have. These could include skills like critical thinking, influencing, problem solving or flexibility. Interviewers often want to hear about challenges you've had, not just about times when everything went smoothly, so be ready with examples such as how you resolved conflict in your team or dealt with someone who was not working well. This will demonstrate that you can handle difficult situations.

When preparing for the job interview, read the job description carefully for the required skills and abilities and try to recall situations where you had to use these skills. Then use the STAR technique when talking about these examples:

- **Situation** – Give details about the context of your example and what you were trying to do.
- **Task** – Describe your responsibilities and the challenges you faced.
- **Action** – Describe what steps you took to deal with the situation.
- **Result** – Talk about the end result and how you contributed to this outcome.

Avoid: Going in unprepared and having to think up examples, or saying you've never faced any challenges at work.

3. Hypothetical questions

What would you do if you had a different opinion from your boss about how to do something?

How would you deal with a large volume of work with several staff members off work?

What would you do if you had to introduce a new policy that you knew was going to be unpopular in your team?

Hypothetical interview questions are similar to competency questions except that instead of asking you to talk about an experience you've had in the past, they present you with an imaginary situation that you might face in your new job.

This might seem difficult to prepare for, but remember that your answers are meant to demonstrate the skills needed for the job. When preparing for the interview, consider the qualities that the interviewer might be looking for, qualities like conflict management, time management or people skills. Then think about how you can demonstrate those qualities in a range of situations. Start with situations that you've experienced and move on to other possible situations that you might encounter in the role you're applying for.

Avoid: Going off-topic, changing the subject and not answering the original question.

Whatever type of questions they ask, interviewers want to find the right person and are keen to give you the opportunity to demonstrate what you can do. With some preparation, you can show them that you're the perfect fit for the job.

Are the sentences true or false?

	Answer	
1. The main message in this article is that you should spend time preparing for a job interview if you have one coming up.	True	False
2. If the interviewer asks you about yourself, you should make sure all the information you give is relevant to the job.	True	False
3. You should avoid talking about personal things like your hobbies.	True	False
4. If the interviewer asks you about an item on your CV, you can give more details and say how it is connected to the job you're applying for.	True	False

- | | | |
|---|------|-------|
| 5. The STAR technique can be used to organise your story when answering competency questions. | True | False |
| 6. You should never talk about difficulties you've experienced at work. | True | False |
| 7. You can't prepare for hypothetical interview questions. | True | False |
| 8. It helps to think about the skills and qualities that the interviewer is looking for when answering interview questions. | True | False |

Put the questions in the correct groups.

Describe a difficult decision you've made.	Why do you want this job?	What would you do if a team member was performing badly?
Tell us about a time when you led a team to achieve an objective.	What would you do if your project goals suddenly changed?	Do you consider yourself successful? Why?
What would you do if you became dissatisfied with your job?	What motivates you?	Tell us about a new skill that you've learned for work.

Common interview questions	Competency questions	Hypothetical questions

MÓDULO 2

Antes de leer el texto:

1. ¿Qué sabe del lenguaje de programación Java?
2. ¿Qué diferencia hay entre Java y otros lenguajes?
3. Pre-visualice el texto que sigue. Elabore una hipótesis de contenido a partir del título, subtítulos y tipografía empleada.
.....
.....
.....
4. ¿Cómo está organizado el texto? ¿Qué función cumple cada sección?
.....
.....
.....

Java vs Other Programming Languages

<https://www.techgeekbuzz.com/blog/java-vs-other-programming-languages/>

Last updated on July 12, 2022

Java is a popular general-purpose programming language. **Also**, it is the fastest **growing** and most reliable programming language. Owned by Oracle, over 3 billion devices all over the world leverage the power of Java. This hints that the trend of Java will not end, not in the foreseeable future. Java has several uses that include:

- Making web apps,
- **Developing** video games,
- Building mobile apps,
- Putting together commercial websites, and so on.

The object-oriented programming language has a broad scope as it can be used in the development of any kind of project.

What Makes Java Different from Other Programming Languages?

Java has some desirable features that make it a great asset to developers. It is platform-independent, which means once you have written Java code, you can run the code on another platform (**operating** system), and the code will run smoothly with no modification.

Java Virtual Machine (JVM) is needed to run any Java Program. JVM executes bytecode, and CPU executes JVM, and all JVMs work the same, thus, making it execute similarly.

Moreover, Java is an object-oriented programming language, which makes it possible to divide complex problems into smaller sets by **creating** objects. This not only increases code reusability but also makes code maintenance easy.

The earlier versions of Java were slow, but with the improvement in JVMs, Java has become faster than other popular programming languages like C++, PHP, and Python.

Another reason Java gets preference among the developers is the availability of many libraries. Hundreds of classes and libraries are available in the ready-to-use Java packages, which is one of the best features of the object-oriented programming language.

So, there is no doubt that Java is a strong language and is better than many other programming languages. But Java also has a few flaws, which are often overlooked due to the wide features it provides. There are so many programming languages that are popular these days. Let's have a look at them and see how they are different from Java. Also, we will also discuss the additional features these programming languages provide.

Java vs. Other Programming Languages

Java vs. C++

C++ is an imperative, general-purpose programming language, which is widely used for competitive **programming**. It runs on many platforms like macOS, UNIX, Linux, and Windows. Though Java is derived from C++, the two differ in many ways. However, Java and C++ are both object-oriented programming languages.

- Java is platform-independent, which can run on any platform, while C++ is platform-dependent.
- C++ supports multiple inheritances, while Java does not support multiple inheritances. Instead, it relies on the concept of interfaces to implement the same.
- C++ is used for system programming.
- Unlike Java, C++ is an extension of the C language.
- C++ is interactive with hardware, but Java is not.
- Supports for the Goto statement are available in C++ and not in Java.

The important difference between the two popular programming languages is that Java has automatic garbage collection, but C++ does not have this feature. Because of this, all the objects have to be destroyed manually in C++.

Also, the standard libraries of these two languages are slightly different. C++ has a simple, standard library, while Java has a standard cross-platform library that is also well-equipped.

Java vs. Python

Python is a powerful high-level programming language. It is an easy-to-use scripting language that supports object-oriented programming the same as Java. However, Python is a dynamically-typed language, which means that there is no requirement for an explicit declaration of the variables before **using** them.

On the contrary, Java is statically-typed. Also, Python is generally appreciated by the new programmers as the Python code is comparatively simple and short in comparison to Java.

In Java, you have to define each variable, but there is no such requirement in Python. It lets you focus on the problem rather than the syntax. Although Python has in-built data types, the popular programming language is not fully equipped for high-level projects such as Java.

Java vs. PHP

PHP is used widely as a server-side scripting language. It is ideal for **creating** web applications. The web scripting language allows developers to create dynamic content that interacts with databases. Moreover, PHP and Java are very different languages.

While PHP is a server-side scripting language, Java is a general-purpose programming language. PHP is a dynamically-typed language (like Python) while Java is statically-typed, where the type is checked at compile time. If a developer uses PHP code, it runs on the server, but in Java, if the client computer does not have **Java Runtime Environment (JRE)**, the code won't execute. There is no such issue with PHP.

Java vs. Ruby

Ruby is a flexible, pure object-oriented programming language. The syntax of Ruby is far more similar to that of C and Java. Hence, it is easy for a Java developer to learn Ruby. **Although** both the programming languages are similar, the key difference between them is that Java translates its code into virtual machine code, which runs fast in comparison to the interpreted Ruby code.

Java is faster than Ruby. Nonetheless, Ruby code is shorter and easy to maintain, which attracts many developers to use it. Interestingly, Java and Ruby, when used together, complement each other. Although Ruby and Java are similar to each other, Ruby cannot by any means serve as a replacement for Java.

Conclusion

So, here we conclude our take on Java vs. other **programming** languages. We compared Java with C++, Python, PHP, and Ruby programming languages, and we clearly get an idea that undoubtedly Java will gain the upper hand. **Because** Java provides all the features that a software engineer needs, it will be a priority always.

With the evolving features among all other languages, there are also chances that developers will shift to new technologies like Python or PHP. However, Java is still a robust option.

Choosing a programming language basically depends on the fact that how efficient and easy to use it is. **Selecting** the right programming language from the very start is important because once a technology is adopted for a project, and the project is initiated, it's tough to shift to another language.

Ahora lea el texto y responda las siguientes preguntas:

- 5) ¿Cuál es la fuente textual?
- 6) ¿Qué tipo de texto es?
- 7) Identifique en el texto: las definiciones de Java, C++, Python, PHP y Ruby.
Interprételas.
- 8) ¿A qué se refiere que Java es independiente de la plataforma?
- 9) ¿Qué función cumple el JVM?
- 10) ¿Cuál es la función más importante de Java? Explique.
- 11) Identifique y explique: las características de: Java, C++, Python, PHP y Ruby.
- 12) Enumere las diferencias entre:
 - a) Java y C++,
 - b) Java y Python,
 - c) Java y PHP,
 - d) Java y Ruby.
- e) Subraye las oraciones que contengan elementos comparados. Trasládelas al castellano e indique qué tipo de comparación se presenta en cada una de ellas. ¿Qué se compara en cada una de ellas?

- f) Localice en el texto un verbo modal de certeza, uno de posibilidad, de obligación, imposibilidad. Transcríbalos y luego traslade al español las oraciones que los contiene.
- g) Analice los ocho conectores lógicos recuadrados en el texto: indique qué tipo de conectores son y sintetice las dos ideas que cada uno de ellos relaciona.



Antes de terminar este módulo vamos a analizar las palabras terminadas en ING. Las palabras terminadas con el sufijo *-ing* son muy comunes en inglés. Según su posición dentro de la oración, pueden cumplir una función sintáctica y comunicativa distinta.

Casos de palabras terminadas en -ing

1) FUNCIÓN VERBAL: BE + VERBO BASE + ING = ANDO/ENDO/IENDO. Esta *ing* representa el aspecto progresivo del verbo en inglés y describe una acción incompleta.

Ejemplos:

a) Researchers **are now trying** to speed up circuitry functions through the use of superconductivity.

b) The object **is moving** and so it has the capacity to work.

2) FUNCIÓN NOMINAL: (PREMODIFICADOR) + SUSTANTIVO NÚCLEO -ING + (POSTMODIFICADOR). Esta *-ing* representa un sustantivo.

Ejemplos:

a) **Designing** software programmes is a major task in IT.

b) The **manufacturing** of integrated circuits started in the late 1950s when the first integrated circuits were developed.

3) FUNCIÓN ADJETIVA: PREMODIFICADOR DE UN SUSTANTIVO: -ING + SUSTANTIVO NÚCLEO. Esta *-ing* representa un adjetivo.

Ejemplos:

a) *Diodes let current through in one direction. This is useful if the input signal is an **alternating** current.*

b) *An optical mouse uses a pair of **light-emitting** diodes (LEDs).*

4) FUNCIÓN ADJETIVA: POSTMODIFICADOR DE UN SUSTANTIVO: SUSTANTIVO NÚCLEO + -ING. Esta *-ing* se traduce como “que + verbo conjugado”.

Ejemplos:

a) *Resistors are used to reduce the current **flowing** through other components.*

b) *The upgraded software program **containing** more features requires professional help to be installed.*

5) FUNCIÓN ADVERBIAL: (BY) + VERBO + ING. Esta *-ing* normalmente anuncia el modo o manera de hacer algo.

Ejemplos:

a) *You can vary the amount of resistance in a circuit by **using** a variable resistor.*

b) *By **updating** your Android apps, you can have access to the latest features and improve app security and stability.*

6) FUNCIÓN ADVERBIAL TEMPORAL: WHEN/WHILE + VERBO + ING. Esta *-ing* se traduce como verbo conjugado en el tiempo presente.

Ejemplos:

a) *When **working on** Microchip's instrumentation amplifiers (INAs), engineers can improve data acquisition applications.*

b) *The company provides services to help engineers and developers make electronic hardware projects while **offering** cost-effective manufacturing resources.*

7) FUNCIÓN ADVERBIAL CAUSAL de EFECTO: Esta *-ing* generalmente está precedida de una coma, después de una cláusula, y se traduce como “lo cual provoca/causa + sustantivo” o “causando + sustantivo / provocando que”.

Ejemplos:

a) *Thermal management under high heat flux is crucial, **developing** high-power light-emitting diode (LED) applications.*

b) *Stretchable electronics is a new emerging class of electronics, **enabling** a range of new applications, for example, cyber skin for robotic devices.*

8) FUNCIÓN COMO INFINITIVO: (PREPOSICIÓN) + VERBO + -ING. Esta ING se traduce como verbo infinitivo, aunque en algunos casos también se puede traducir como sustantivo.

Ejemplos:

a) ***Moving** things can be described as energy which enables them to do work.*

b) *Audio amplifiers are used for **amplifying** an electrical signal.*

c) *Instead of **downloading** that software version, you should check first that it is its latest update.*

h) Complete el siguiente cuadro con la traducción en el casillero correspondiente de las palabras terminadas con ING según su función en el texto.

Base+ING	Función verbal	Función Infinitivo	Función Nominal	Función Adjetivo (pre – post)	Función adverbial
<i>Growing</i>					
<i>Developing</i>					
<i>Operating</i>					

Creating					
Programming					
Using					
Creating					
Programming					
Selecting					

ACTIVIDADES

Lea las siguientes oraciones y complete el siguiente cuadro con la traducción en el casillero correspondiente de las palabras terminadas en “-ING” según su posición sintáctica. La primera está resuelta a modo de ejemplo.

1. The meaning of a program is defined by its semantics.
2. *Object-oriented (OO) programming* provides a model in which the program is a collection of objects that interact with each other by passing messages that transform their state.
3. The compiling process translates a source program into the language of a computer. Later, the resulting *machine code* can be run on that computer.
4. The ideas discussed in this chapter provide a basis for describing the syntax of a complete programming language.
5. Programming in a higher-level language usually reduces type errors, since compilers and run-time systems are designed to identify them.
6. **Type Rule:** *All declared variables must have unique names.* This rule can be enforced by checking that every distinct pair of variables in a declaration list must have mutually different identifiers.
7. When debugging, it is important to have a methodological approach to help you find where things are breaking down.
8. This chapter introduces the semantics of programming languages from an informal point of view. An operational approach to semantics is used, highlighting the semantic features of several popular languages. The semantics of expressions, assignments, conditionals, branching, and looping statements are considered, along with those of input/output and exception handling.

9. Although many new artificial intelligence (AI) programs are focusing on automating software testing and debugging, job seekers still need these skills.
10. *Pair programming* is a technique that involves two developers working at one workstation to complete a task.
11. Before proceeding with this tutorial, you should have a basic understanding of Computer Programming terminologies.
12. Object-oriented programming makes code understandable by encapsulating moving parts. Functional programming makes code understandable by minimizing moving parts.

BASE + ING	FUNCIÓN VERBAL	INFINITIVO	FUNCIÓN NOMINAL	FUNCIÓN ADJETIVA (PRE MODIFICADOR)	FUNCIÓN ADJETIVA (POST MODIFICADOR)	FUNCIÓN ADVERBIAL
1.meaning	_____	_____	El significado	_____	_____	_____
2.passing						
3.compiling						
3.resulting						
4.describing						
5.programming						
6.checking						
7.debugging						
8.highlighting						
8.branching						
8.looping						
9.focusing						
10.working						
11.proceeding						
12.encapsulating						
13.minimizing						

14.moving						
-----------	--	--	--	--	--	--

Computer Language Compiler - *Last retrieved - (2017)*



In May 1952, at Harvard University in Cambridge, Massachusetts, Grace Murray Hopper completed her work on the first computer language compiler, software that forms an important link between human language and binary code, thus enabling more efficient, less error-prone computer programming.

Hopper held a professorship in mathematics at Vassar College and a doctorate in the field. In 1943, she enlisted in the Navy's Women Accepted for Voluntary Emergency Service (WAVES) program. **She** had the opportunity to work on a team at the Bureau of Ordinance Computation Project at Harvard University's Cruft Laboratory with Howard Aiken, **who** was the principal engineer on the Mark I computer. She was the third person to ever work on **this machine**, which is considered America's first large-scale automatic digital computer. Hopper ended up writing a five-hundred-page manual to the Mark I that would define standard concepts for operating computers, and she would write programs for the Mark II and III series computers to follow.

Hopper continued as a research fellow at Harvard until 1949, when she took a position as mathematician at Eckert-Mauchly Computer, developers of the UNIVersal Automatic Computer (UNIVAC). This machine was the first computer built for commercial use and used magnetic tape, instead of punch cards, to input and store data.

Hopper was tasked with developing a program for the UNIVAC that would make it easier for programmers to translate English instructions into language that the computer would understand. Instead of writing out programs in binary code (a cumbersome set of ones and zeros), a "compiler" permitted a programmer to use English commands to compose code. **This approach** allowed programmers to build code that was less prone to error and took much less time. More critically, by simplifying programming, a compiler would expand interest in computing beyond the province of mathematicians or computer scientists.

Her first compiler was finished in 1949. This “A-O” program used symbolic mathematical code to stand for binary code combinations. Specific call numbers would attach to a collection of programming routines stored on magnetic tape. To perform a calculation, the user would use a call number, **which** the computer would find on tape, bring over, and do the task.

In 1952, she completed a true breakthrough in computer programming, the “B-O” or “Flow-Matic” compiler. Her goal was to create a tool useful in business settings, such as automatic billing and payroll calculation. The Flow-Matic is considered the first computer language compiler for the English language, and the compiler to program the UNIVAC I and II machines. Flow-Matic also became the basis for developing the universal computer language COBOL (COmmon Business-Oriented Language) in the late 1950s.

1. Lea el texto y conteste las siguientes preguntas:

a) ¿Qué tema anuncia el título?

.....
.....
.....

b) ¿Qué sucedió en mayo del 1952?

.....
.....
.....

c) ¿Qué es el Mark I?

.....
.....
.....

d) ¿Qué tarea se le asignó a Hopper para UNIVAC?

.....
.....
.....

e) ¿Cuándo se terminó su primer compilador?

.....
.....
.....

f) Describa el programa “A-O”.

.....

.....

.....

.....

g) ¿Cuál era el objetivo del compilador “B-O”?

.....

.....

.....

.....

2. ¿A qué se refieren las siguientes frases o palabras?

- 1) She:
- 2) Who:
- 3) This machine:
- 4) This approach:
- 5) Which:

3. Transcriba cuáles son los conectores lógicos detallados a continuación y explique, en castellano y con sus propias palabras, cuáles son las ideas relacionadas por los mismos:

a. Un marcador lexical de *consecuencia*:

.....

.....

.....

.....

b. Un marcador lexical de *adición*:

.....

.....

.....

.....

c. Un marcador lexical de *ejemplificación*:

.....

.....

.....

.....

.....

4. Complete el siguiente cuadro con la traducción en el casillero correspondiente de las palabras terminadas en **-ING** según su función en el texto.

Base+ING	Función verbal	Función infinitivo	Función nominal	Función adjetivo (pre – post)	Función adverbial
Enabling (1° párrafo)					
Programming (1° párrafo)					
Operating (2° párrafo)					
Writing out (4° párrafo)					
Simplifying (4ª párrafo)					
Billing (6° párrafo)					

Biografía profesional – Professional Biography:

¿Qué es una biografía profesional?

Es un resumen conciso y relevante de la trayectoria laboral, logros y habilidades de una persona en el ámbito profesional. A diferencia de una biografía personal, que puede abarcar una variedad de aspectos de la vida de una persona, una biografía profesional se centra específicamente en su experiencia laboral, educación, especializaciones y cualquier otro detalle relevante para su carrera o campo de actividad. Esta biografía suele utilizarse en páginas web, redes sociales profesionales, presentaciones, currículos, resumen de LinkedIn y otros contextos donde es importante destacar la credibilidad y la experiencia de la persona en un entorno laboral específico.

Estructura

En una breve biografía, estará limitado a solo unas pocas oraciones o un breve párrafo. Es importante que incluya solo la información básica sobre usted. Uno de los principales objetivos de una biografía es enfatizar sus logros. Esto proporcionará al lector una idea general de sus antecedentes. Esta información no necesita ser demasiado detallada. Además, una biografía está escrita en “tercera persona”. Esto significa que debe evitar usar “yo” y presentarse como si estuviera leyendo la

biografía de otra persona. Las siguientes oraciones proporcionan ejemplos del formato apropiado.

Empiece con su información básica e incluya lo siguiente:

- Nombre y apellidos
- Su función actual
- Su objetivo final
- Su mayor logro

Algunos ejemplos de biografías profesionales - Some examples of professional bio:

1. Corporate executive:

"Mary Johnson is the Chief Financial Officer at Global Enterprises, where she oversees financial planning, risk management, and strategic investments. With over 20 years of experience in corporate finance, Mary has been instrumental in driving the company's growth and profitability. She holds an MBA from Harvard Business School and is a Certified Public Accountant (CPA). Mary is also a member of the American Institute of CPAs."

2. Academic:

"Dr. Robert Lee is a Professor of Environmental Science at Green University. His research focuses on climate change mitigation and sustainable development. With over 30 years of experience in academia, Dr. Lee has authored over 50 peer-reviewed articles and received several prestigious awards, including the National Science Foundation Grant. He earned his Ph.D. from Stanford University."

3. Creative Professional:

"Hi, I'm Emma, a freelance graphic designer and illustrator. I specialize in creating vibrant, eye-catching designs that help brands stand out. With over 10 years of experience, I've worked with clients ranging from startups to established companies. In my free time, I enjoy painting, traveling, and spending time with my rescue dog, Max."

4. Professional Biography of Mark C. Hersam



Materials Science and Engineering, Northwestern University 2220 Campus Drive, Evanston, IL 60208-3108 USA

Tel: 847-491-2696; Fax: 847-491-7820

E-mail: m-hersam@northwestern.edu

WWW: <http://www.hersam-group.northwestern.edu/>

Mark C. Hersam is currently a Professor of Materials Science and Engineering, Chemistry, and Medicine at Northwestern University. He earned a B.S. in Electrical Engineering from the University of Illinois at Urbana-Champaign (UIUC) in 1996, M.Phil. in Physics from the University of Cambridge in 1997, and a Ph.D. in Electrical Engineering from UIUC in 2000. In 1999, he also performed research at the IBM T. J. Watson Research Laboratory under the support of an IBM Distinguished Fellowship. His research interests include nanofabrication, scanning probe microscopy, semiconductor surfaces, and carbon nanomaterials. As a faculty member, Dr. Hersam has received several awards including the Beckman Young Investigator Award, NSF CAREER Award, ARO Young Investigator Award, ONR Young Investigator Award, Sloan Research Fellowship, Presidential Early Career Award for Scientists and Engineers, TMS Robert Lansing Hardy Award, AVS Peter Mark Award, ECS SES Research Young Investigator Award, MRS Outstanding Young Investigator Award, and five Teacher of the Year Awards. In recognition of his early career accomplishments, Dr. Hersam was directly promoted from assistant professor to full professor with tenure in 2006. In 2007, Dr. Hersam co-founded NanoIntegris, which is a start-up company focused on supplying high performance carbon nanomaterials. Dr. Hersam is a Fellow of MRS and SPIE in addition to serving as the Chair of the AVS Nanometer-scale Science and Technology Division and as Associate Editor of *ACS Nano*.

5. David Lee, Software Engineer

David Lee is a highly skilled Software Engineer with 5 years of experience in developing scalable web applications. Currently working at ABC Tech, he has a strong background in JavaScript, Python, and Ruby on Rails. David holds a Bachelor's degree in Computer Science from the University of California, Berkeley. He has contributed to open-source projects and led a team in developing a high-traffic e-commerce platform. David is committed to continuous learning and is particularly interested in artificial intelligence and machine learning.

Match the definitions (a–f) with the vocabulary (1–6).

Vocabulary

Definition

- | | |
|-------------------------------|---|
| 1. brand loyalty | a. evidence of all the successes you have had in the past |
| 2. brand awareness | b. customers' habit of always buying the same product from the same company |
| 3. sector | c. to encourage people to be interested |
| 4. a proven track record | d. how much customers know about a product |
| 5. to drive engagement | e. helping to achieve a plan |
| 6. strategic | f. an area of economic or professional activity |

Reading text: A professional profile

David Beddingfield
Digital Marketing Manager
Summary

Over the past six years I have held various roles in the field of digital marketing and communication and currently lead a small team in the Fast-Moving Consumer Goods (FMCG) sector. I have a proven track record of planning and implementing successful digital marketing campaigns across a range of channels.

Experience
Digital Marketing Manager

Krunch Ltd
Jan 2017 – Present

My role involves strategic focus on all areas of digital marketing within the company and working closely with the main marketing and communications teams. I design and lead all our digital marketing campaigns.

Digital Marketing Executive

Krunch Ltd
June 2014 – Dec 2016

I was responsible for overall social media strategy. I worked on many campaigns to drive engagement with the customers of our breakfast cereals and bars to build brand loyalty. I also led on three key product launches.

Digital Marketing Assistant

Food2Grab Ltd
Sept 2013 – May 2014

In this role I developed my existing knowledge and skills in the fields of social media and consumer goods. I supported the design of multi-channel marketing campaigns to promote brand awareness of our healthy snack range.

Tips

1. Your online profile needs to be concise, engaging and convincing.
2. State a clear title/role under your name.
3. Your summary should be in short paragraphs.
4. Think about who you want to read your profile and write in an appropriate style for them.
5. Add links to examples of your work if you have them.
6. Check it carefully and get feedback from a friend or colleague.

Are the sentences true or false?

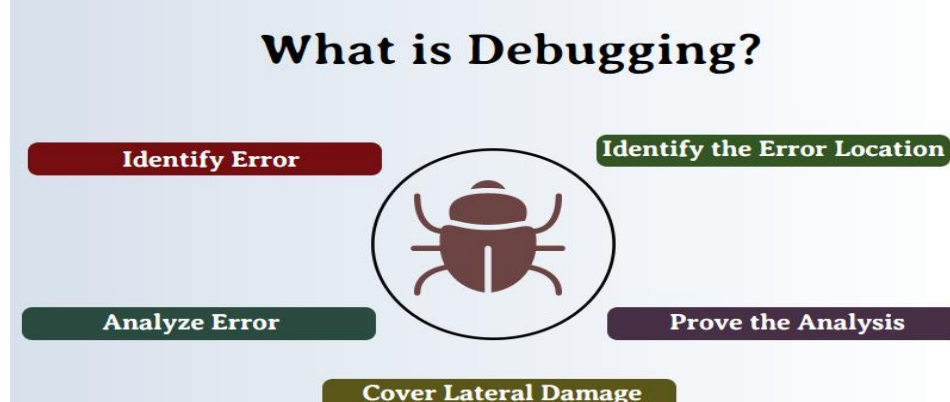
	Answer	
1. David Beddingfield specialises in marketing products via the internet.	True	False
2. David manages people.	True	False
3. He joined Krunch Ltd in 2017.	True	False
4. He is responsible for creating new digital marketing campaigns.	True	False
5. The objective of the breakfast cereal and bars campaign was to tell people about the new product.	True	False
6. The objective of the healthy snack range campaigns was to make sure people knew about the product.	True	False

Final task- Tarea final:

Escribe un párrafo sobre tu propia biografía corta, haciendo referencia a tu vida profesional y académica. Utiliza como modelo los ejemplos vistos.

MÓDULO 3

<https://www.educba.com/what-is-debugging/>



Introduction to Debugging

Debugging is the process of **finding** bugs i.e. error in software or application and fixings them. Any software program or product which is **being** developed undergoes through various steps – testing, troubleshooting, maintenance in a different environment. These software programs or product contains some errors or bugs. These bugs need to be removed from the software to develop bug-free software. Debugging is nothing but a process that many software testers used to find those bugs and fix them. **Debugging** refers to finding bugs, analyzing, and fixing them. This process takes place when the software fails **due to** some errors or software performs an unwanted thing. Debugging looks simple **but** it is a complex task **as** there is a need to fix all errors at every stage of debugging.

Why do we need Debugging?

Any software which is developed needs to be bug-free before releasing or **going into** the market. As there is much competition in the market, every organization wants to be at the top. This is possible if your software is bug-free and the customer is happy with your software. The customer becomes happy if he or she does not find any error while using the software. To make a customer happy, software needs to be bug-free and resolved by using the **debugging** process. This is the reason why every organization must do the debugging process before releasing it into the market.

Process of Debugging

The different stages involved in the process of debugging are:

1. Identify Error

Finding errors at an early stage of the development process can save a lot of time. If you make any mistake in identifying an error, it leads to wasted developing time. Commonly, production errors reported by users are hard to interpret, and sometimes

the information we get is misleading. Errors or bugs found at the user site are not easy to find. Finding the error is necessary to save time and avoid errors at the user site.

2. Identify the Error Location

After finding the error correctly, you need to go through the code to find the perfect spot where the error is found. It helps to fix the problem or debug faster. In this stage, you need to focus on identifying the error instead of understanding it.

3. Analyze Error

In the third stage, you need to use a bottom-up technique from the error location and analyze the code. This helps you in finding the bugs or errors. It helps in checking around the error for other errors to be found and to find about the risks of entering any collateral damage in the bug fixing. This stage is important as solving one error may lead to another error.

4. Prove the Analysis

Once the identified error has been analyzed, you need to focus on finding a few more errors that may appear on the application. This step is about writing automated tests where you need to write the test cases with the help of a test framework. In case you run all test cases and pass the test, you can move to the next stage, or else, you look into or resolve the test case that doesn't pass the test.

5. Cover Lateral Damage

In this stage, you have to do unit **testing** of all the code where you make the changes. If all test cases pass the test, then you can move to the next stage or else you have to resolve the test case that doesn't pass the test.

6. Fix and validate

The last stage is fixing all the errors and running all the test scripts to make sure that they all pass.

Advantages of Debugging

The debugging has several advantages:

- It reports an error condition immediately when it is found. This helps in the earlier detection of an error and makes the process of software development simple and unproblematic.
- It also offers information about data structures and enables easy interpretation.
- With the help of the debugging process, the developer can eliminate complex one-use testing code to save a lot of time in software development.

Lea el texto y luego conteste en castellano las preguntas sobre la información desarrollada en el texto.

- a. ¿A qué se refiere el término “*debugging*”? ¿Cuándo y para qué se realiza?
 - b. ¿Cuál es la razón por la cual las organizaciones deben realizar el proceso de depuración antes de lanzarlos al mercado?
 - c. Nombre las etapas que se realizan en el proceso de depuración. Explique cada una brevemente.
 - d. ¿Cuáles son las ventajas de la depuración?
- 1) ¿A qué se refieren las siguientes palabras recuadradas en el texto?
- a. Which (1° párrafo):
 - b. This (párrafo 2):
 - c. It (párrafo 4):
 - d. This (párrafo 5):
 - e. This stage (párrafo 5):
 - f. This step (párrafo 6):
- 2) Traslade al castellano los siguientes bloques nominales subrayados en el texto:
1. Any software program or product:
 2. The different stages involved in the process of debugging:
 3. a bottom-up technique:
 4. the earlier detection of an error:
 5. information about data structures:
 6. complex one-use testing code:

7. Busque y subraye una definición en el texto. Luego traslade la oración que la contiene.
8. En el texto hay cuatro conectores lógicos resaltados en negrita. ¿De qué tipo son y qué ideas relacionan?
- a) i.e.: conector de.....
Idea 1:
Idea 2:
- b) due to: conector de
idea 1:
idea 2:
- c) but: conector de
Idea 1:
Idea 2:
- d) as: conector de
idea 1:
idea 2:
9. En el texto hay cinco verbos modales diferentes: ¿Cuáles son y qué función cumple cada uno de ellos? Luego, traduce las oraciones que los contienen.
10. Complete el siguiente cuadro con la traducción en el casillero correspondiente de las palabras terminadas en -ING resaltadas en negrita según su función en el texto.

Base+ING	Función verbal	Función infinitivo	Función nominal	Función adjetivo (pre – post)	Función adverbial
<i>Finding (p. 1)</i>					
Being (p.1)					
Debugging (p.1)					
Going into (p.2)					
Debugging (2)					
Finding (p.3)					
Testing (p.7)					

LA VOZ PASIVA

Observe las siguientes oraciones extraídas de un texto sobre cómo funciona Python y señale la acción y quien la realiza la acción en cada una de ellas:

- 1) Python interprets code, but C++ compiles code.
- 2) Code is interpreted by Python, but code is compiled by C++.
- 3) JavaScript was created by Brendan Eich in 1995 during his time at Netscape Communications.

Analice las oraciones 2) y 3). Escriba verdadero (V) o falso (F) en los siguientes enunciados:

- a) El sujeto de la oración realiza la acción. _____
- b) El sujeto de la oración recibe la acción. _____
- c) El agente del verbo está mencionado. _____
- d) La oración está en tiempo pasado. _____
- e) La oración está en tiempo presente. _____
- f) El verbo terminado en -ED indica que la oración está en tiempo pasado. _____
- g) Tienen la misma estructura verbal. _____

En inglés, en general, la oración simple tiene esta estructura:

SUJETO + VERBO + OBJETO DIRECTO o INDIRECTO + ADVERBIAL

Ejemplo: *The Company selects the best applicants for the job.*

El sujeto *The Company* ejecuta la acción de seleccionar y es el sujeto activo. Pero si queremos focalizar nuestra atención en la acción, usamos la **voz pasiva**.

Ejemplo: *The best applicants for the job **are selected** by the company.*

La estructura de la voz pasiva es la siguiente:

SUJETO PASIVO+ (ser o estar) BE (IS/ARE/WAS/WERE/WILL (etc.) BE) + verbo lexical en la forma PARTICIPIO PASADO+ (BY) AGENTE

La voz pasiva permite enfatizar una acción o un estado. El sujeto de la acción no tiene relevancia, no se conoce o se asume que todo el mundo lo conoce.

- ✓ Es más normal encontrar en español formas con 'se', por ejemplo: 'se habla', 'se alquila' o verbos en plural como 'venden', 'compran'. Por ejemplo:

German is spoken here / *Aquí se habla alemán*

When was this house built? / *¿Cuándo se construyó (fue construida) ésta casa?*

A lot of songs have been written about love / *Se han escrito muchas canciones sobre el amor*

- ✓ En ocasiones, la oración en voz pasiva presenta el **agente** (también llamado “complemento agente”) que permite identificar quién realiza, realizó, etc. la acción. El mismo es anunciado por la preposición **by**. Por ejemplo:

This house was built by my mother. / *Esta casa fue construida por mi madre.*

Washington was bombed by Pakistan. / *Washington fue bombardeado por Pakistán.*

- ✓ Se usa **ESTAR + PARTICIPIO** para expresar un estado o un resultado.
This desk is made from wood. / *Este escritorio está hecho de madera.*

Verb Tense	Structure	Example
PRESENT SIMPLE	am/are/is + pp	Spanish is spoken here.
PRESENT CONTINUOUS	am/are/is being + pp	Your questions are being answered.
FUTURE (WILL)	will be + pp	It'll be painted by next week.
FUTURE (GOING TO)	am/are/is going to be + pp	Terry is going to be made redundant next year.
PAST SIMPLE	was/were + pp	We were invited to the party, but we didn't go.
PAST CONTINUOUS	was/were being + pp	The hotel room was being cleaned when we got back from shopping.
PRESENT PERFECT	have/has been + pp	The President of America has been shot .
PAST PERFECT	had been + pp	When he got home he found that all of his money had been stolen .
FUTURE PERFECT	will have been + pp	Our baby will have been born before Christmas.

Ejemplos:

- Experience **is required** for the job.
- Competent students **are usually selected** for the best internships.
- The vacant jobs **were advertised** in yesterday's newspaper.
- Tim **will be hired** if his application **is admitted**.
- CVs **are carefully read** by Human Resources recruiters.
- You can succeed if you **are given** an opportunity.

Actividad:

- Busque en el primer texto del módulo ejemplos de voz pasiva y traslade la oración completa al castellano.
 - Lea las siguientes oraciones, identifique la estructura pasiva y trasládelas al castellano.
- Keywords are predetermined names that can be used to identify things in your code.

- b. Functions are blocks of code that run when they are invoked. They are the workhorse for your program and are used to perform operations and manipulations on your code.
- c. *Conditional statements* allow you to perform checks on whether a block of code should be executed or not.
- d. All problems in computer science can be solved by another level of indirection.
- e. Higher-level programming languages are generally divided for convenience into compiled languages and interpreted languages.
- f. The **front end** checks whether the program is correctly written in terms of the programming language syntax and semantics. Here legal and illegal programs are recognized. Errors are reported, if any, in a useful way. Type checking is also performed by collecting type information.
- g. The output of a compiler that produces code for a virtual machine may or may not be executed on the same platform as the compiler that produced it. For this reason, such compilers are not usually classified as native or cross compilers.
- h. Java was originally developed by **James Gosling** at *Sun Microsystems* (which has since been acquired by Oracle Corporation) and released in 1995 as a core component of Sun Microsystems' Java platform.

- i. The C programming language was originally developed for and implemented on the UNIX operating system by Dennis Ritchie.
- j. Operating systems were first developed in the late 1950s to manage tape storage.
- k. Any piece of data that is stored in a computer's memory must be stored as a binary number. That includes characters, such as letters and punctuation marks. When a character is stored in memory, it is first converted to a numeric code. The numeric code is then stored in memory as a binary number.

Elaborar resúmenes de textos

Cómo elaborar un resumen de un texto

Autora del material: Lorena Andrade Benavides

I. ¿Qué es un resumen de texto?

El **resumen de un texto** es un **escrito de carácter informativo** que tiene como propósito principal presentar una **síntesis de los contenidos** que el autor del texto original transmitió a través del escrito.

II. ¿Para qué sirve el resumen de un texto en el marco de la carrera?

El resumen de un texto es una **herramienta de aprendizaje** que te ayudará a facilitar la comprensión de los contenidos, revisados en los distintos cursos de la carrera, por medio de la **síntesis** de la información que entregan los textos que el profesor (a) asigna para leer en cada sesión. Estos textos funcionan como el apoyo principal para el docente en sus clases, pues transportan el **contenido teórico** en que se basa la asignatura.

III. ¿Cuál es la estructura del resumen?

Etapas del resumen	Función
Introducción	Presenta el tema y el propósito del autor del texto que se resume.
Desarrollo	Explica los temas centrales del texto original .
Conclusión	Ofrece una síntesis de los contenidos tratados en el resumen.

Al momento de escribir tu resumen, recuerda seguir los siguientes pasos:

ESCRITURA	Paso 1	Presentar el propósito del autor del texto que se resume por medio de una oración simple ¹ . Por ejemplo: El objetivo del autor de este texto es explicar las actividades básicas de los seres humanos.
	Paso 2	Agrupar y categorizar las ideas principales de cada párrafo, según las características que compartan por medio de hiperónimos ² . Por ejemplo: si en un párrafo del texto, el autor sostiene la siguiente idea: "...el modo de percepción de las autoridades, del personal y de los usuarios en torno al sistema de salud..."; esta idea se puede categorizar a partir del
		concepto actores sociales , término que funciona como hiperónimo de los participantes nombrados. Por tanto, esta idea se podría sintetizar de la siguiente manera: "Los modos de percepción de los actores sociales en torno al sistema de salud...".
	Paso 3	Luego de agrupar las ideas, debes ordenarlas por medio de marcadores discursivos de tiempo para explicar los procesos que realiza el autor del texto original. Por ejemplo: En primer lugar , el autor define y caracteriza el concepto de salud y, en segundo lugar , explica el rol de la población en este tema.
	Paso 4	Enseguida, debes explicar los procesos que realiza el autor. Para ello, puedes relacionar las ideas por medio de conectores . Por ejemplo: El autor define salud como un bien propio y humano, ya que todos los seres humanos lo desean de forma natural.
	Paso 5	Para terminar, sintetiza las ideas principales de cada párrafo y comienza a escribir tu resumen de acuerdo a la estructura presentada en el apartado III de la guía.

Verifica si cada etapa del resumen cumple su función convencional. Con el fin de cumplir este propósito, lee tu resumen y, luego, responde las preguntas que se presentan a continuación.

Etapas del resumen	Preguntas que ayudan a comprobar la función de esta
Introducción	¿Presenta el tema del resumen de manera sintética por medio de una oración simple? ¿Da cuenta del propósito comunicativo del autor del artículo sin dejar ver la subjetividad del autor (a) del resumen?
Desarrollo	¿Explica los temas más importantes del artículo? ¿Estos temas se desarrollan a partir de una idea principal por párrafo?
Conclusión	¿Deja ver una síntesis de los contenidos tratados en el resumen de manera objetiva? ¿La síntesis es una parafraseo de las ideas dichas por el autor del artículo?

INTEGRACIÓN

7 Reasons Why Beginner Programmers Should Study PHP Programming Language

Published: 3 December 2021

<https://robots.net/tech/php-programming-language/>

PHP is one of the primary programming languages developers use to create websites on the internet. It is one of the most flexible and easy-to-use programming languages available. As a server-side language, PHP runs scripts on the website's server instead of the client's web browser. PHP is also open-source, which means anyone who wants to use, modify, or distribute it can do so for free.

"PHP" stands for Hypertext Preprocessor. Initially, it was called "Personal Home Page" because it was used to create personal home pages. **But** the programming language evolved, acquiring new features and capabilities with each upgrade. **Thus**, the developers working on the language changed its name into something more appropriate to its function.

We also need to emphasize that PHP is mainly used for programming, not coding. As a programming language, it's used to create executable machine programs instead of simply converting code from one language to another, as is the case with coding.

PHP can be used to create various web applications, starting from standard web pages to e-commerce platforms to GUI applications. In fact, reports indicate that over 80 percent of websites across the globe rely on PHP. Some websites that run on PHP programming include WordPress, Yahoo, Facebook, Flickr, MailChimp, and Wikipedia.

JavaScript, Python, and C+ seem to be getting all the attention from online communities these days. But that doesn't mean that the PHP programming language is already doomed to obscurity. It remains highly relevant and popular with a wide range of applications. So before you drop PHP off your list of things to learn, consider the following reasons why you should:

1. PHP Provides an Excellent Foundation for Programming

For beginner coders and those just diving into WordPress development, PHP is one of the best places you can start. There are only two things that you need to get started with PHP: a development environment to run your PHP scripts (e.g., XAMPP, WAMP) and a code editor to write the code (e.g., Notepad ++, Atom). Once you have your tools ready, you can already begin writing your PHP script. The task would be even faster and easier to accomplish with a framework.

2. It Is Easy to Learn

PHP scripting is easy to learn, and its syntax is quite similar to Java, C, C++, and Perl. This means a person with experience in these languages should find it relatively easy to adapt to PHP. Like Java, PHP is an interpreted language; it's ready to run

and doesn't require compilation into machine instructions. It provides the instructions for servers to interpret at runtime, making it less complicated than other programming languages.

Most experienced programmers also consider PHP as a stable programming language. A stable language is one whose fundamentals remain the same for a very long time. It allows programmers to use their knowledge and skills on that particular knowledge for a long time without the risk of everything becoming obsolete. On the other hand, you have an unstable language like JavaScript that receives a bombardment of updates and extensions, making it hard for programmers to keep up with.

3. PHP Is Accessible to All

PHP is an open-source program, which means it's available for download and use at zero cost. It's also known for modification and distribution without special permission. Both beginners and experienced programmers can contribute to language development, and any developer can use the language to create web pages.

Another factor that plays into PHP's accessibility has to do with the fact that it supports a large selection of mature frameworks. A framework is simply a program or code library that writes most of the code on your behalf and only leaves crucial decision points for you to code. Think of frameworks as customized versions of PHP. Developers create these frameworks for other developers, which means they are accessible for free. Examples of the most popular PHP frameworks include Laravel and Symfony.

4. Learning PHP Helps You Become a Well-Rounded Programmer

As a general-purpose programming language, PHP can be used to accomplish a wide range of tasks. Some examples you can achieve with PHP include: performing mathematical calculations, collecting user information, interacting with MySQL databases, creating graphics, etc. But more importantly, you can use PHP to create a wide selection of website types, including content management systems, GUI-based applications, information-based websites, and so on. You will also find it mentioned quite a lot in online forums and programming tutorials.

5. The Language Is Secure

PHP is a server-side language, processing the code left to the server instead of on the client's web browser. **Moreover**, the servers are responsible for storing all the data associated with the website, and servers are protected with layers upon layers of security.

The original PHP developers designed it to work hand-in-hand with HTML, which is the standard format for all web pages on the internet. This works because an end-user or client sends a request to the server for a copy of an HTML page. The server then runs the PHP code first and sends the output in HTML format back to the requesting party. In other words, PHP is a precursor for generating the final HTML file that we see on websites.

The most crucial aspect of server-side scripting is that the client-side never sees the script, only the output of the script sent by the server. Moreover, you can also protect scripts on the server side with passwords. You can also configure your website to run all HTML files with PHP to protect the scripts from client browsers. The fact that your script runs on the server side offers more security for your website than the opposite.

6. There Is a Huge Community of Developers

PHP has been around for more than 25 years, and it has remained an open-source program throughout that time. During that time, it has managed to acquire a robust network of developers who work to enhance the capabilities of the language, essentially pushing its boundaries.

The way this works is developers have access to the source code. Their goal is to tweak the code to incorporate additional features and capabilities into the program. They also try to fix errors with the language.

It's important to note that the developers working to improve PHP aren't paid to work on the program. But they do it anyway **because** they could use the extra features and capabilities. Practically anyone with some basic knowledge of programming can pitch in.

In addition, there is also a large repository of resources and information online about PHP. If you need help regarding PHP, many blogs, forums, video tutorials, and e-books offer help with PHP programming.

7. PHP Is a Well-Established Programming Language

PHP is a legacy programming language, which means it's been around for a long time now. A Danish-Canadian programmer named Rasmus Lerdorf created the original PHP code back in 1994. At the time, only one percent of websites on the internet had been built using PHP. Since then, developers have increasingly turned to PHP as their go-to language for web servers, and even now, we see it being used to back up popular websites.

- 1) Lea el texto y luego haga un resumen del texto en español teniendo en cuenta la guía.
- 2) ¿A qué se refieren las siguientes palabras?
 - Which (1° párrafo):
 - That (5° párrafo):
 - This (7° párrafo):
 - Their (8° párrafo):
 - Which (13° párrafo):
- 3) Traslade al castellano los siguientes bloques nominales:
 - a server-side language:

- Most experienced programmers:
 - Both beginners and experienced programmers:
 - a wide selection of website types, including content management systems, GUI-based applications, information-based websites:
- 4) Analice los conectores resaltados en negrita en el texto. Indique qué tipo de conectores son y sintetice las dos ideas que conectan cada uno de ellos.
- 5) Analice los siguientes verbos modales que aparecen en el texto. ¿Qué modalidad presentan? Luego, traslade al castellano la oración que los contiene.
- NEED
CAN
WOULD
SHOULD
WILL
COULD
- 6) Subraye cuatro oraciones pasivas en el texto y luego traslade al castellano las oraciones que los contenga.
- 7) Complete el siguiente cuadro con la traducción en el casillero correspondiente de las palabras resaltadas y subrayadas en el texto, terminadas en –ING según su función en el texto.

Base+ING	Función verbal	Función infinitivo	Función nominal	Función adjetivo (pre – post)	Función adverbial
<i>Programming</i> (p.3)					
Following (p.5)					
Diving (p.6)					
Making (p.7)					
Collecting (p.11)					
Working (p.17)					

MÓDULO 4

Lea el siguiente texto y resuelva las actividades en español.

Will AI Replace Programmers? Navigating the Future of Coding

By Andrea Siedsma

When addressing AI and its impact on programming, it's essential to recognize its incredible capabilities. AI has already revolutionized various industries. However, amidst its capabilities, some significant limitations deserve attention.

One primary concern is the potential for what's called "hallucination." **This** means that while Generative AI can create genuine content, there's a risk of generating misleading or inaccurate information, which raises questions about reliability and precision.

Another crucial point is the legal aspects surrounding the use of Generative AI. As this technology advances, issues like copyright infringement, intellectual property rights, and ethical considerations become more complex, **calling** for clear guidelines and regulations to navigate these challenges effectively.

Additionally, the quality of the output heavily depends on the input data. If the data is insufficient or biased, it can lead to skewed outputs and inaccurate predictions. This underscores the significance of robust data collection and preprocessing techniques to ensure the accuracy and integrity of Generative AI's results.

Norman McEntire, an instructor for Extended Studies, founded Servin Corporation in 1995 to provide computer training, consulting, and software development with a focus on Android, iOS, Java, Embedded C, Linux, and Machine Learning. With more than 25 years of experience, he has designed both hardware and software at all levels, from embedded Linux to enterprise software.

The biggest change the industry veteran has seen in the field is the impact of machine learning and artificial intelligence. Notably, he said, **it** has a considerable influence on both computer hardware and computer software and more importantly, can have a significant impact on programmer's workflows.

"The possibilities are vast and exciting, from automated code generation to intelligent debugging tools, from predictive analytics to personalized coding assistance," says McEntire.

According to McEntire, programmers must understand fundamental coding knowledge to collaborate with AI effectively and to communicate these technical concepts to stakeholders. Although he feels current and future programming careers are not at risk, there are ways AI can affect your workflow, McEntire said.

One way is by streamlining routine tasks through automation. Far from being a threat, AI is a valuable collaborator for programmers. Many routine and repetitive tasks can be automated, allowing you to focus on more complex problem-solving and creative aspects of coding. Take, for example, code optimization. AI algorithms can analyze vast amounts of data to suggest improvements, enhancing the efficiency of your code. This saves you time and allows you to allocate your efforts to more meaningful aspects of programming.

Additionally, AI can assist in the debugging process. Advanced debugging tools powered by AI can quickly identify and rectify errors, making the development cycle faster and more

efficient. These efficiencies allow programmers to become more like strategists, guiding AI tools to achieve optimal results and having more time for problem-solving.

By embracing AI as a collaborative partner, programmers can enhance their capabilities and productivity, ultimately becoming more proficient and well-rounded.

The key to staying ahead of the curve is to keep learning, McEntire said. McEntire currently teaches one of Extended Studies newest programming courses, Embedded Machine Learning. In today's tech landscape, leveraging AI empowers programmers to achieve unprecedented productivity. AI can quickly help with tasks like providing syntax suggestions, software maintenance, code review, and offering insights into best coding practices.

"By using AI as an assistant, you can accomplish more tasks in a shorter period. Everyone works faster, so we have more time to focus on solving more problems and producing innovative technology. With AI, existing programmers can produce new and improved code faster. And others who perhaps were hesitant to program can now use AI to help with coding," says Norman.

- 1) Realice un resumen de no más de 150 palabras sobre el texto. No incluya el contenido preguntado en la siguiente pregunta
- 2) ¿Cuál es la clave para ser un buen programador hoy en día?
- 3) ¿A qué se refieren los siguientes términos recuadrados en el texto?
 - a) **This (párrafo 2):**
 - b) **It (párrafo 6):**
- 4) Busca relaciones lógicas y contesta las preguntas que se encuentran a continuación
 - a) En el tercer párrafo hay un conector de causa/razón. ¿Cuál es el conector y que ideas relaciona? Conector: causa:

consecuencia:
 - b) En el párrafo 12 hay un conector de adición. Especifica cuál es y qué ideas conecta. Conector:
Idea 1:

Idea 2:

5) **Resalte con color** en el texto al menos dos **oraciones condicionales**. Indica su tipo y luego traslade al castellano la oración completa:

6) **Resalte con color** dos oraciones pasivas del texto. Transcriba el bloque verbal pasivo. Luego tradúcela en forma completa.

Bloque verbal pasivo:

Traducción:

.....
.....

Bloque verbal pasivo:

Traducción:

.....
.....

7) Indique la función del -ing de la palabra marcada en el texto y escribala en castellano en el lugar correcto

Base+ING	Función verbal	Función Infinitivo	Función Nominal	Función Adjetivo (pre – post)	Función adverbial (¿Cuál?)
CALLING (P. 4)					
SOLVING (P. 12)					
PROVIDING (p.15)					

ORACIONES CONDICIONALES (CONDITIONAL SENTENCES)

CONDICIONAL 0

En el caso del condicional 0, la relación de causa efecto es factual. Muestra una relación lógica y esperable.

Ambas clausulas estarán en presente simple y el conector IF debe estar seguido por la idea que representa la causa o condición y la otra clausula, es decir el efecto o resultado, puede encontrarse en primer término o en segundo término.

Ejemplo:

If you heat ice, it melts (or ice melts if you heat it)

Causa: si calientas hielo

Efecto: se derrite

CONDICIONAL 1

Este tipo de oraciones condicionales muestra típicas relaciones de causa-efecto o acciones probables en el tiempo. La cláusula asociada al IF que será la causa o condición estará en presente simple y la otra cláusula estará típicamente en futuro simple es decir se utilizará el verbo modal WILL. Estas cláusulas también pueden invertirse en orden.

Ejemplo:

If you drop it, it will break.

CAUSA: Si lo sueltas

EFEECTO: se romperá

CONDICIONAL 2

Este tipo de oraciones condicionales muestra relaciones de causa-efecto imaginarias, improbables o hipotéticas. La cláusula asociada al IF que será la causa o condición estará en pasado simple, pero será interpretado en el modo subjuntivo y la otra cláusula estará típicamente en condicional simple es decir se utilizará el verbo modal WOULD. Estas cláusulas también pueden invertirse en orden.

Ejemplo:

If I touched the engine right now, I would get burnt

CAUSA: Si tocara/tocase el motor ahora

EFEECTO: me quemaría

CONDICIONAL 3

Este tipo de oraciones condicionales muestra relaciones de causa-efecto imposibles. La cláusula asociada al IF que será la causa o condición estará en pasado pluscuamperfecto (pasado perfecto) y la otra cláusula estará típicamente en condicional compuesto es decir se utilizará el verbo modal WOULD acompañado por HAVE. Estas cláusulas también pueden invertirse en orden.

IF-CLAUSE	MAIN CLAUSE
Zero Conditional (always)	
<i>Present Simple</i>	<i>Present Simple</i>
First Conditional (present or future time)	
<i>Present Simple</i>	<i>Future Simple (will + infinitive)</i>
<i>Present Simple</i>	<i>Modal verb (can/must) + infinitive</i>

<i>Present Simple</i>	<i>Imperative</i>
<i>Present Continuous</i>	<i>Future Simple (will + infinitive)</i>
Second Conditional (present or future time)	
<i>Past Simple</i>	<i>Conditional Simple (would + infinitive)</i>
<i>Past Simple</i>	<i>Modal verb (could) + infinitive</i>
Third Conditional (past time)	
<i>Past Perfect (had + past participle)</i>	<i>Conditional Perfect (would + have + past participle)</i>
<i>Past Perfect (had + past participle)</i>	<i>Modal verb (could) + have + past participle</i>

Ejemplo:

If the scientist had made the procedure properly, he would not have spoiled the sample

CAUSA: Si el científico hubiese hecho el proceso correctamente

EFFECTO: no habría arruinado la muestra

- 1) Identifique en el texto leído al comienzo del módulo y subraye una oración condicional en el texto, establezca de qué tipo es y luego trasládelo al castellano.

Aquí un resumen de las oraciones condicionales:

.....
.....

Otras conjunciones usadas en oraciones condicionales:

if - si

whether - si

unless - a menos que

provided / providing - siempre que, mientras

as long as - siempre que, mientras

in case – por si acaso, por si, en caso de que

If

If you had called me, I would have gone to the football match with you.

I'd go to the gym if I had more free time.

Unless

Unless introduce una condicional negativa, casi siempre de los tipos cero y uno.

I'll go running unless it rains = I'll go running if it doesn't rain.

I'll give you a hand unless you don't want me to = I'll give you a hand if you want me to.

As long as / so long as

Este conjunto de frases es más enfático que *if*, **hace más hincapié en la necesidad de que se cumpla la condición** para que la consecuencia también lo haga.

We'll get there on time as long as we leave now.

As long as we don't make many mistakes, we should win this match easily.

Providing (that) / provided (that)

Estas conjunciones condicionales en inglés tienen básicamente el mismo significado que las anteriores, aunque **son algo más formales**.

We will get there on time providing that we leave now.

Provided that we do not make many mistakes, we should win this match easily.

In case

I'll take my umbrella if it rains.

En este caso, lo que queremos decir es que me llevaré el paraguas únicamente si llueve (es decir, en caso de que llueva). Si no hay lluvia, no hay paraguas.

I'll take my umbrella in case it rains.

ACTIVIDADES:

Lea las siguientes oraciones condicionales, establezca a qué tipo pertenecen y trasládelas al castellano:

- 1) A language is *statically typed*, if the types of all variables are fixed when they are declared at compile time.
- 2) **Definition of polymorphic:** A function or operation is *polymorphic* if it can be applied to any one of several related types and achieve the same result.
- 3) If he had studied, he would have passed his exam.

- 4) If the compiled program can run on a computer whose CPU or operating system is different from the one on which the compiler runs, the compiler is known as a cross-compiler.
- 5) As long as you are an experienced programmer, this website is intended for everyone who wishes to learn the C programming language.
- 6) If I had enough money, I would buy a new computer.
- 7) Most of the time, as a programmer, you will be writing a programme to solve a particular problem. If you don't enjoy solving problems, you won't enjoy being a programmer.
- 8) If I had asked him, he would have helped me.
- 9) How can you finish the project if you don't have a computer?
- 10) Unless you hurry up, you won't finish writing the characteristics of Java.
- 10) In case the programmer had a mobile phone, he would call all his friends.
- 11) Some languages (like C) have a rather weak type system, so it is possible to make serious errors if you are not careful.

Presentar un pitch

Tips para tu presentación final grupal

Un "pitch" es una presentación breve y persuasiva, generalmente de pocos minutos, para exponer una idea, producto o servicio a un público objetivo. Su objetivo principal es captar el interés del oyente y, si es posible, convencerlo para que realice una acción específica, como invertir, comprar o colaborar.

Para explicar un pitch, puedes seguir estos pasos:

1. Define tu objetivo:

¿Qué quieres lograr con este pitch? ¿A quién vas a dirigir tu presentación?
Asegúrate de tener claro el propósito y el público objetivo antes de comenzar.

2. Identifica la idea principal:

¿Cuál es la esencia de tu propuesta? ¿Qué valor único ofreces? El pitch debe ser conciso y enfocado en la idea principal.

3. Estructura tu mensaje:

- **Introducción:** Captar la atención desde el primer momento. Puedes usar una pregunta impactante, una estadística interesante o una afirmación sorprendente.
- **Problema o necesidad:** Asegúrate de que el público entienda el problema que vas a resolver o la necesidad que vas a cubrir.
- **Solución:** Explica claramente cómo tu propuesta resuelve el problema o satisface la necesidad.
- **Propuesta de valor:** ¿Por qué tu solución es la mejor opción? ¿Qué beneficios ofrece?.
- **Llamada a la acción:** ¿Qué quieres que haga el público después de escuchar tu pitch?.

4. Usa un lenguaje claro y conciso:

Evita jerga técnica y utiliza un lenguaje que sea fácil de entender para tu público objetivo.

5. Practica tu discurso:

Ensaya varias veces para que el pitch fluya de forma natural y te sientas seguro al presentarlo.

6. Adapta tu presentación:

Si tienes la oportunidad, personaliza tu pitch para que se ajuste al público específico.

7. Sé breve pero persuasivo:

Recuerda que el pitch debe ser conciso y directo, pero también debe transmitir confianza y entusiasmo.

Puedes ver estos videos para ayudarte:

<https://www.youtube.com/watch?v=jYWF64Um7pw>

<https://www.youtube.com/watch?v=1uVxa9XQYG8>

Tips para hacer una PPT atractiva

- **Elegir colores o imágenes atractivas o relevantes al tema elegido. La presentación no debe ser puramente textual.**

- Elegir una fuente con un tamaño visible y que sea fácil de leer como Arial o Calibri.
- No incluir demasiado texto en cada diapositiva sino tan solo palabras o frases claves.
- Utilizar un fondo de diapositivas consistente y que no llame demasiado la atención. No utilizar demasiadas diapositivas. Solo las necesarias.
- Usar colores de alto contraste entre texto y fondo.
- Pueden obtener modelos de diapositivas de SLIDES TO GO
- Si se incluyen vídeos deben estar embebidos en una diapositiva y ser muy breves.
- La presentación debe tener como máximo 15 diapositivas: la primera debe presentar el tema y los miembros del grupo y la última agradeciendo la participación a la audiencia.
- Si incluyen gráficos es importante que sean claros y además sean explicados.
- Al comienzo o al final se puede incluir un punteo de temas a modo de índice.

A la hora de la presentación oral

- Tener la presentación en un back up en caso de que exista algún problema de compatibilidad. Pueden usar medios físicos como un Pen Drive o nubes.
- Presentar a los participantes y el tema con una diapositiva o de manera oral.
- Aclararle a la audiencia que al final de la presentación se responderán preguntas.
- Respetar los tiempo dados y para esto es necesario practicar la presentación con un cronometro.
- No leer de las dispositivas (o lo menos posible)

ANEXOS

EXAMEN MODELO 1

- 1) Pre-visualice el siguiente texto.
- 2) Lea el título del texto. ¿Qué tema se puede inferir a partir del mismo?
- 3) Elabore una hipótesis a partir del título, el copete y la tipografía empleada.

Science News

from research organizations

Hardware, software advances help protect operating systems from attack

The **operating** system (OS) is the backbone of your computer. If the OS is compromised, attackers can take over your computer -- or crash it. Now researchers

at North Carolina State University have developed an efficient system that utilizes hardware and software to restore an OS if it is attacked.

At issue are security attacks in which an outside party successfully compromises one computer application (such as a Web browser) and then uses that application to gain access to the OS. For example, the compromised application could submit a "system call" to the OS, effectively **asking** the OS to perform a specific function. However, instead of a routine function, the attacker would use the system call to attempt to gain control of the OS.

"Our goal is to give the OS the ability to survive such attacks," says Dr. Yan Solihin, an associate professor of electrical and computer **engineering** at NC State and co-author of a paper **describing** the new system. "Our approach has three components: attack detection; security fault isolation; and recovery."

The concept is to take a snapshot of the OS at strategic points in time (such as system calls or interrupts), when it is **functioning** normally and then, if the OS is attacked, to erase everything that was done since the last "good" snapshot was taken -- effectively going back in time to before the OS attack. The mechanism also allows the OS to identify the source of the attack and isolate it, so that the OS will no longer be vulnerable to attacks from that application.

The idea of **detecting** attacks and re-setting a system to a safe state is a well-known technique for **restoring** a system's normal functions after a failure, but this is the first time researchers have developed a system that also incorporates the security fault isolation component. This critical component prevents the OS from succumbing to the same attack repeatedly.

The concept of **taking** snapshots of the OS and using it to replace the OS if it is compromised was previously viewed as impractical, since taking these snapshots and **running** such a system significantly slowed computer operating speeds. "But we've developed hardware support that allows the OS to incorporate these survivability components more efficiently, so that they take up less time and energy," Solihin says. The researchers say the survival system takes up less than 5 percent of the OS's operating overhead.

The paper, "Architectural Framework for Supporting Operating System Survivability," was co-authored by Solihin and former NC State Ph.D. student Xiaowei Jiang. The paper will be presented Feb. 16 at the IEEE International Symposium on High-Performance Computer Architecture in San Antonio, Texas. The research was supported, in part, by the National Science Foundation.

- 4) Ahora lea el texto en detalle y responda las siguientes preguntas:
- 5) ¿A qué hace referencia el sistema operativo de la computadora?
- 6) ¿Qué han desarrollado los investigadores de la Universidad estatal de Carolina del Norte?

- 7) ¿Quién podría bloquear tu computadora y qué probable consecuencia tendría esto?
- 8) ¿De qué forma el atacante podría controlar el sistema operativo?
- 9) ¿Cómo logrará el Dr. Solihin que el sistema operativo logre sobrevivir a los ataques de seguridad?
- 10) ¿Cómo funciona el mecanismo de aislamiento de fallas de seguridad?
- 11) ¿Cómo funciona el mecanismo de la captura de pantalla al sistema operativo?
- 12) Traslade al castellano los siguientes bloques nominales:
- three components: attack detection; security fault isolation; and recovery:
.....
 - a snapshot of the OS at strategic points in time:
.....
 - a well-known technique for restoring a system's normal functions after a failure:
.....
.....
 - Architectural Framework for Supporting Operating System Survivability:
.....
- 13) ¿A qué se refieren las palabras recuadradas en el texto?
- **It** (1° párrafo):
 - **Our** goal (3° párrafo):
 - **That** (6° párrafo):
- 14) ¿Qué modalidad marcan los verbos subrayados? Traslade al castellano sólo la frase verbal completa.
- **Can** (1° párrafo):

- **Could** (2° párrafo):
- **Would** (2° párrafo):
- **Will** (4° párrafo):

15) Re-lea las oraciones que contienen conectores en negrita en el texto. Identifique qué tipo de relación lógica establecen y luego sintetice las dos ideas que cada uno de ellos conecta.

a) En el segundo párrafo hay un conector de contraste. Identifica el conector (en inglés) y sintetiza las dos ideas que une.

Conector: _____

Idea 1:

Idea 2:

b) En el cuarto párrafo hay un conector de propósito. ¿Cuál es? Completa las ideas.

Conector: _____

Propósito:

De qué:

c) Hay un conector de causa/razón en el sexto párrafo. Identifica el conector y completa las ideas.

Conector: _____

Causa:

Consecuencia:

16) Transcriba al castellano la siguiente oración. Indique qué tipo de comparación presenta.

"We've developed hardware support that allows the OS to incorporate these survivability components more efficiently, so that they take up less time and energy,"

.....
.....
.....

17) Complete el siguiente cuadro con la traducción en el casillero correspondiente de las palabras terminados en *-ING* según su función en el texto.

BASE + ING	FUNCIÓN VERBAL	INFINITIVO	FUNCIÓN NOMINAL	FUNCIÓN ADJETIVA (PRE MODIFICADOR)	FUNCIÓN ADJETIVA (POST MODIFICADOR)	FUNCIÓN ADVERBIAL
Operating (párrafo 1)						
Asking (párrafo 2)						
Engineering (párrafo 3)						
Describing (párrafo 3)						
Functioning (párrafo 4)						
Detecting (párrafo 5)						
Restoring (párrafo 5)						
Taking (párrafo 6)						
Running (párrafo 6)						

EXAMEN MODELO 2

Lea el siguiente texto y resuelva las actividades en español

Understanding JIT compiler (just-in-time compiler)

Java JIT Compiler: An Overview

The key of java power "Write once, run everywhere" is bytecode. The way bytecodes get converted to the appropriate native instructions for an application has a huge impact on the speed of an application. These bytecodes can be interpreted, compiled to native code or directly executed on a processor whose Instruction Set Architecture is the bytecode specification. Interpreting the bytecode, **which** is the standard implementation of the Java Virtual Machine (JVM), makes execution of programs slow. To improve performance, JIT compilers interact with the JVM at run time and compile appropriate bytecode sequences into native machine code. When using a JIT compiler, the hardware can execute the native code, as opposed to having the JVM interpret the same sequence of bytecode repeatedly and incurring the penalty of a relatively lengthy translation process. **This** can lead to performance gains in the execution speed, **unless** methods are executed less frequently. The time that a JIT compiler takes to compile the bytecode is added to the overall execution time, and could lead to a higher execution time than an interpreter for executing the bytecode if the methods that are compiled by the JIT are not invoked frequently. The JIT compiler performs certain optimizations when compiling the bytecode to native code. **Since** the JIT compiler translates a series of bytecode into native instructions, it can

perform some simple optimizations. Some of the common optimizations performed by JIT compilers are data-analysis, translation from stack operations to register operations, reduction of memory accesses by register allocation, elimination of common sub-expressions etc. The higher the degree of optimization done by a JIT compiler, the more time it spends in the execution stage. **Therefore**, a JIT compiler cannot afford to do all the optimizations that is done by a static compiler, both because of the overhead added to the execution time and because it has only a restricted view of the program.

How does it work?

The Just-In-Time (JIT) compiler is a component of the Java Runtime Environment that improves the performance of Java applications at run time. Java programs consists of classes, **which** contain platform neutral bytecode that can be interpreted by a JVM on many different computer architectures. At run time, the JVM loads the class files, determines the semantics of each individual bytecode, and performs the appropriate computation. The additional processor and memory usage during interpretation means that a Java application performs more slowly than a native application. The JIT compiler helps improve the performance of Java programs by compiling bytecode into native machine code at run time.

The JIT compiler is enabled by default, and is activated when a Java method is called. The JIT compiler compiles the bytecode of that method into native machine code, compiling it "just in time" to run. When a method has been compiled, the JVM calls the compiled code of that method directly instead of interpreting it. Theoretically, if compilation did not require processor time and memory usage, compiling every method could allow the speed of the Java program to approach that of a native application.

JIT compilation does require processor time and memory usage. When the JVM first starts up, thousands of methods are called. Compiling all of **these** methods can significantly affect startup time, even if the program eventually requires processor time and memory usage.

1. Escriba un resumen del texto en no más de 150 palabras y no menos de 80-

2. Traduzca los siguientes bloques nominales:

Some of the common optimizations performed by JIT compilers:

.....

data-analysis, translation from stack operations:

.....

processor time and memory usage:

.....

3. ¿A qué hacen referencia las siguientes palabras?

- Which (1° párrafo):

- This (1° párrafo):

- These (3° párrafo):

4. ¿Qué tipo de relación lógica establecen los siguientes conectores? Sintetice las dos ideas que cada uno de ellos conecta:

- Unless (párrafo 1): es un conector de:

Condición:

Resultado:

- Since (párrafo 1): es un conector de:

Causa:

Consecuencia:

- Therefore (párrafo 1): es un conector de:

Causa:

Consecuencia:

- Transcribe del texto un **verbo modal**. Indica su función y luego traduce al español la oración que lo contiene:

-verbo modal:

- función:

5. Complete el siguiente cuadro con la traducción en el casillero correspondiente de las palabras terminadas con ING según su función en el texto.

Base+ING	Función verbal	Función infinitivo	Función sustantivo	Función adj. (pre o post modificador)	Función adverbial
using (párrafo					

1)					
compiling (párrafo 2)					
interpretin g (párrafo 1)					
interpretin g (párrafo 3)					
compiling (párrafo 3)					

- Subraye dos oraciones pasivas en el texto y luego tradúcelas.
- Identifique una doble comparación. Traslade la oración al castellano.
- Identifique una oración condicional. Establezca de qué tipo es y trasládela al castellano.

CONECTORES

ADICIÓN	Traducción orientadora	LISTADO Y ENUMERACIÓN	Traducción orientadora	RESUMEN Y GENERALIZACIÓN	Traducción orientadora	ALTERNATIVA	Traducción orientadora
And And also As well as Both....and Not onlybut also In addition Also Moreover Besides Furthermore What is more	Y Y también Así también como Tanto como No sólo.....sino también Además También Además Más aún Lo que es más	First / Firstly / At first Second / Secondly Next Last/Lastly Finally To begin with In the second place To conclude Then	Primero / en primer lugar /al principio Segundo / en segundo lugar A continuación Por ultimo Finalmente Para empezar En segundo lugar Para terminar Luego	In short In a word To sum up In all All together More generally In other words	En síntesis En una palabra En suma “ “ Más generalmente En otras palabras	Or Or else Either.....or Neither..... nor Whetheror Instead Otherwise	O O sino O bieno Ni ni O o / ya sea.....o En cambio De otro modo
CONTRASTE	Traducción orientadora	RESULTADO CONSECUENCIA	Traducción orientadora	RAZON/CAUSA	Traducción orientadora	PROPÓSITO	Traducción orientadora

		INFERENCIA	a		a		a
But However/None theless Nevertheless / Still / Notwithstanding g Yet Although While On the one hand... On the other hand In spite of /Despite Whereas Even though /Though	Pero Sin embargo No obstante “ Aún así Aunque/si bien Mientras que Por un lado... Por otro lado A pesar de (que) Mientras que Aún cuando	Therefore So / Thus / Then Consequently As a result of Accordingly Hence As a consequence	Por lo tanto Entonces /Así/ De este modo En consecuen cia Como resultado de Por consiguiet e Por lo tanto Como consecuen cia	Because Because of Since For Due to As Owing to	Porque A causa de Ya que /dado que Pues Debido a Ya que, como Debido a	So that /In order that So as to / In order to	De manera que Para / con el objeto de
CONDICIÓN	Traducción orientadora	CONDICIÓN	Traducción orientadora	EJEMPLIFICACIÓN	Traducción orientadora	CLARIFICACIÓN	Traducción orientadora
If Unless (So) As long as Even when Providing (that)	Si A menos que Siempre que Incluso cuando Siempre que “	Supposing (that) On condition (that) Assuming (that) Suppose (that) Provided (that)	Suponiend o (que) A condición (de que) Asumiendo (que) Suponga (que) Siempre que	For example For instance e.g Such as Like	Por ejemplo “ “ Tal como Como	That /This is (to say) i.e Namely Viz	Es decir “ Especialme nte / O sea A saber
TEMPO- ESPACIALES	Traducción orientadora						
When While At the same time After that Before that As soon as	Cuando Mientras Al mismo tiempo Después de éso/ello Antes de éso/ello Tan pronto como						